# GHANA SMART SERVICE OPERATIONS OPTIMIZER

*Comprehensive Project Report: Architecture, Design, & System Implementation*

DCIT 204/308 Joint Data Structures & Algorithms Semester Project
Stack: Java & PostgreSQL | Scope: Shai-Osudoku District, Ghana

## 1. Project Overview & Assignment

The Ghana Smart Service Operations Optimizer is a joint Data Structures & Algorithms semester project (DCIT 204/308) developed by a 15-person team over a 4-week period using Java and PostgreSQL. The foundational non-negotiable requirement of the assignment is that every team member must independently defend one data structure and one algorithm during an oral demonstration—no group submission can cover for an individual who cannot explain their own assigned slot.

This single requirement shapes the entire architecture: 15 people each own exactly one structure and one algorithm, paired so the two components explain each other (e.g., Kruskal's algorithm relies on a disjoint set; a priority heap is consumed by Dijkstra's shortest path).

## 2. Domain & Data Provenance

The system models district hospital service requests—including patient transfers, specimen runs, drug deliveries, and referrals—arriving from CHPS compounds and health centres. It determines what to handle next, who handles it, and by what optimal route, operating under realistic constraints such as limited porters, ambulances, shift times, and degraded roads.

- **Real-World Context:** The geographical and operational setting is the Shai-Osudoku District, Ghana. Locations, roads, and real names are sourced and documented in project data provenance records.
- **Dataset Scale:** The dataset encompasses 50 locations, 100 roads, 300 service requests, and 30 resources (porters and ambulances), fully loaded into a live PostgreSQL database read and written by the running application.
- **Dual-Layer Topology:** The domain features two layers joined at exactly one point: the external network (communities, health centres, referral routes) and the internal hospital layout (wards, theatres, labs, corridors), meeting at the ambulance bay (type = 'gateway').

## 3. Engineering Non-Negotiables

To comply with strict academic grading rubrics and prevent point deductions, all design decisions strictly adhered to the following non-negotiables:

- **No Java Built-in Collections:** Assessed logic completely avoids Java built-in collections like HashMap, PriorityQueue, Stack, and ArrayDeque. Every structure is hand-built and array- or node-backed.

- **Comprehensive Testing:** A minimum of three tests per structure (covering normal input, boundary conditions, and invalid input) is enforced, clearing the 40-test floor.
- **Dynamic Configuration:** No tunable values are hardcoded. Every configuration parameter lives centrally in gsoo.app.Config and derives dynamically from team roster index numbers.
- **Member-Derived Parameters:** At least three algorithm parameters derive from member index numbers via a documented formula (S = 6316, the sum of the last three digits of all 15 index numbers).
- **Empirical Evidence Generation:** Trace tables, counterexamples, and benchmark numbers are generated directly by running real code against the live dataset rather than written manually.
- **Live Database Integration:** The database operates as an active component of the running system rather than a static seed-once file.

## 4. Team Structure & Pod Organization

The 15 team members are organized into three specialized pods of five, each pairing structure-heavy components with graph and optimization components:

| Pod | Focus Area | Slot Assignments |
|---|---|---|
| Pod A | Linear Structures & Elementary Search | A1 (Dynamic Array / Binary Search), A2 (Linked List / Linear Search), A3 (Stack / DFS), A4 (Queues), A5 (Deque / Insertion Sort) |
| Pod B | Trees, Hashing & Indexing | B1 (BST / Quicksort), B2 (AVL / Merge Sort), B3 (B-Tree / Brute Force), B4 (Hash Table / Selection Sort), B5 (Custom Map) |
| Pod C | Graph, Priority & Optimization | C1 (Hash Set / Greedy), C2 (Heap / Dijkstra), C3 (Disjoint Set / Kruskal), C4 (Adjacency List / Prim), C5 (Adjacency Matrix / Knapsack) |

Fenuku Reynolds Elikem (NEMESIS-7), the project author, holds Slot A1 alongside the Architect & Integrator role, managing the shared Graph interface, Config, application entry point, PR reviews, and cross-cutting integration.

## 5. System Architecture & Execution Story

The application (gsoo.app.Main) provides a unified console walkthrough centered around a single 'focus request' loaded from the live PostgreSQL database. Rather than 15 disconnected demos, it tells one cohesive narrative story:

- **Startup & Session Loading:** Connects to Postgres via gsoo.db and DatabaseLoader, loading 50 locations, 100 roads, 30 resources, and 300 requests, and selecting today's focus request.

- **Request Search & Filtering:** Located in the sorted request table via Binary Search (A1) and compared against un-ordered Linear Search (A2).
- **Audit History & Tracking:** Logged via Stack (A3) with real database writes for audit trails and reviewed via Linked List (A2).
- **Queueing & Triage:** Evaluated using FIFO Queues (A4) and Custom Deque (A5) for priority triage.
- **Categorization & Indexing:** Categorized and looked up using Custom Map (B5) and B-Tree indexing (B3).
- **Routing & Graph Traversal:** Analyzed via Adjacency List graph connections (C4), Depth-First Search road closures (A3), and Hash Set reachability tracking (C1).
- **Sorting Comparisons:** Compared across QuickSort (B1), MergeSort (B2) with stable tie-breaking, SelectionSort (B4), and InsertionSort (A5).
- **Assignment & Shift Planning:** Processed through Brute Force assignment (B3), Greedy assignment counterexamples (C1), Kruskal's & Prim's Minimum Spanning Trees (C3, C4), and Dynamic Programming Knapsack shift scheduling (C5).